

ПРИНЦИПЫ ПОСТРОЕНИЯ ПАРАЛЛЕЛЬНЫХ ВЫЧИСЛИТЕЛЬНЫХ СИСТЕМ

Способы параллельной обработки данных

- Под параллельными вычислениями понимаются процессы обработки данных, в которых одновременно выполняются несколько операций компьютерной системы.
- Два основных способа параллельной обработки данных: собственно параллелизм и конвейерные вычисления

Собственно параллелизм

$$S = \sum_{j=0}^N 1 / (a^2 + j^2), \quad S, a \in \mathbb{R}, \quad N \gg 1$$

Оптимизирующий компилятор C/C++: все переменные – в регистрах процессора

Последовательное суммирование на одном процессоре (ядре) x64 с системой команд SSE2: $\approx 16(N + 1)$ тактов процессора.

Пусть имеется $p \in \mathbb{N}$ процессоров с номерами $k = 0, p$

$$S_k = \sum_{j=nk}^{n(k+1)-1} \frac{1}{a^2 + (j+1)^2}, \quad n = \left\lfloor \frac{N+1}{p} \right\rfloor, \quad k = \overline{0, p-2}$$
$$S_{p-1} = \sum_{j=n(p-1)}^N \frac{1}{a^2 + (j+1)^2}, \quad S = \sum_{k=0}^{p-1} S_k$$

Пусть S_k независимо, т. е. параллельно вычисляется на $(k+1)$ -м процессоре: $\approx 16(N+1) / p$ тактов процессора.

Примерно за это время может быть вычислена величина S .

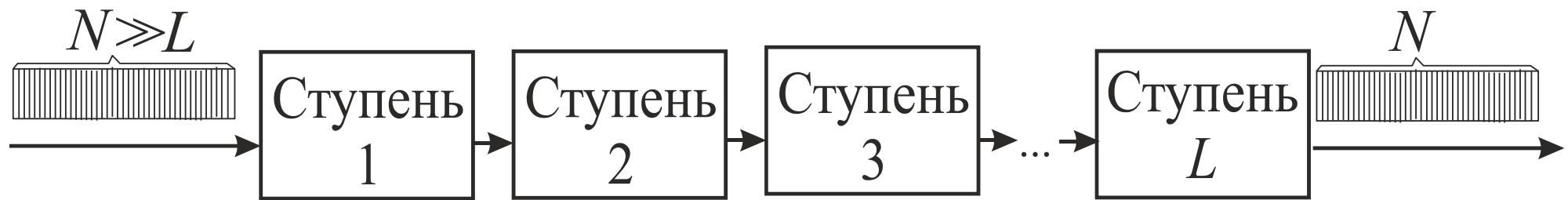
- Более общая ситуация: свертка достаточно длинной последовательности элементов $\{x_j\}_{j=0}^N$, $N \gg 1$, $x_j \in X$ некоторой бинарной операцией $\circ$ в результирующий элемент $S \in X$

$$S = x_1 \circ x_2 \circ x_3 \circ x_4 \circ \dots \circ x_{N-2} \circ x_{N-1} \circ x_N \quad (1)$$

- Для распараллеливания необходима ассоциативность бинарной операции $a \circ b \circ c = (a \circ b) \circ c = a \circ (b \circ c)$. Можно расставить скобки в (1) и разделить вычисления между процессорами
- Планировщики параллельных программ иногда меняют порядок операций. Если бинарная операция коммутативна $a \circ b = b \circ a$, то результат вычисления S будет однозначно определен.
- Суперскалярная архитектура современных процессоров: каждое ядро содержит несколько блоков для операций над данными логического, целого, и вещественного типов, для трансляции виртуальных адресов в физические. По возможности, операции выполняются параллельно.
- Векторные команды процессоров: математические операции выполняются сразу над несколькими парами операндов

Конвейерная модель вычислений

- Пример: сложение пары вещественных чисел двойной точности за 4 такта в блоке процессора x64
- Выполняется последовательность микроопераций: сравнение порядков, выравнивание порядков, сложение мантисс, нормализация и т.д.
- Все микрооперации имеет одинаковую длительность 1 такт.
- В процессе обработки каждой пары входных данных микрооперации задействуются лишь один раз и всегда в одном и том же порядке.
- Если некоторая микрооперация выполнила свою работу и передала ее результаты следующей микрооперации, то для обработки текущего набора данных она больше не потребуется, и, значит, она может начинать обработку следующего набора данных.



- В функциональном устройстве (ФУ) каждую микрооперацию выделим в отдельную часть, называемую ступенью конвейера, и расположим их в порядке выполнения микроопераций.
- Общее число ступеней L обычно называют длиной конвейера.
- Пусть время выполнения всех микроопераций одинаково.
- В первый момент времени входные данные поступают для обработки первой ступенью конвейера.
- После выполнения первой микрооперации первая ступень передает результаты своей работы второй ступени, а сама берет на обработку новые данные и т.д.
- Когда входные данные пройдут через все этапы обработки, на выходе ФУ появится результат выполнения операции.

- Если конвейерное устройство содержит L ступеней, а каждая ступень обрабатывает за одну единицу времени, то время обработки набора данных длины N , т.е. выполнения N независимых операций данным устройством составит $N + L - 1$ единиц времени.
- Если устройство использовать как последовательное, когда для выполнения одной операции оно блокируется на L тактов в монопольном режиме, то время обработки будет NL единиц времени.
- Ускорение:

$$\frac{NL}{N + L - 1} \xrightarrow{N \rightarrow \infty} L$$

Современные процессоры используют т.н. конвейерную модель вычислений: выборка, декодирование и непосредственное исполнение команд выполняются параллельно.

Пути достижения параллелизма

- Независимость функционирования отдельных устройств вычислительной системы. Относится в равной степени ко всем основным компонентам вычислительной системы: к обрабатывающим процессорам, устройствам памяти и к устройствам ввода-вывода.
- Избыточность элементов вычислительной системы. Может осуществляться в следующих основных формах:
 - Использование специализированных устройств: отдельные процессоры для целочисленной и вещественной арифметики; устройства многоуровневой памяти (регистр, кэш-память);
 - Дублирование устройств вычислительной системы: использование нескольких однотипных обрабатывающих процессоров или нескольких устройств оперативной памяти.
- Дополнительная форма обеспечения параллелизма: конвейерная реализация обрабатывающих устройств.

Типы и режимы организации параллелизма

- Многозадачный режим с разделением времени, при котором для выполнения нескольких процессов (т.е. процессов либо потоков ОС) используется единственный процессор.
 - Режим является псевдопараллельным, когда активным (исполняемым) может быть один процесс, а все остальные процессоры находятся в режиме ожидания своей очереди.
 - Может повышать эффективность организации вычислений в случае, когда некоторые процессы находятся в состоянии ожидания (например, происходит ввод данных), а процессор может быть задействован для выполнения другого готового к исполнению процесса.
 - Проявляются многие эффекты, свойственные параллельным вычислениям, и данный режим может быть использован для начальной подготовки параллельных программ.

- Параллельное выполнение, когда в один и тот же момент может выполняться несколько команд обработки данных.
 - Такой режим вычисления может быть обеспечен не только при наличии нескольких процессоров, но и при помощи конвейерных и векторных обрабатывающих устройств.
- Распределенные вычисления. Данный термин обычно применяют для указания параллельной обработки данных, при которой используется несколько обрабатывающих устройств, достаточно удаленных друг от друга, в которых передача данных по линиям связи приводит к существенным временным задержкам.
 - Эффективная обработка данных при подобном способе организации вычислений возможна лишь для параллельных алгоритмов с низкой интенсивностью потоков межпроцессорных передач данных.

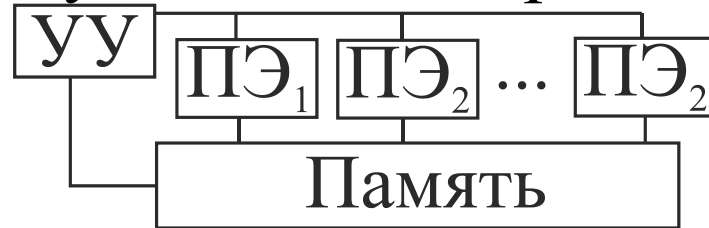
Классификация М. Флинна архитектур вычислительных систем
Предложена в 1966 г. и базируется на понятии потока (stream), под которым понимается последовательность команд или данных, обрабатываемых процессором. (Кэш-память уровня 1 современных процессоров, как правило, содержит 2 части: для хранения команд и для хранения данных.)

На основе анализа числа потоков команд и потоков данных выделяют 4 класса архитектур:

- **SISD** (Single Instruction Stream/Single Data Stream) – системы, в которых существует одиночный поток команд и одиночный поток данных. Последовательная цифровая ЭВМ Дж. фон Неймана

● **SIMD** (Single Instruction Sream/Multiple Data Sream). Система с одиночным потоком команд и множественными потоками данных.

- Одна команда обрабатывает сразу несколько информационных элементов, при этом одно устройство управления синхронизирует работу нескольких процессорных элементов



- Синхронная параллельная обработка данных
- Ранее широко использовались многопроцессорные системы с единым устройством управления (архитектура Cray).
- В пределах отдельных ядер современных процессоров векторные команды для математических операций выполняются сразу над несколькими парами операндов

Процессор/набор команд	Длина векторных операндов			
	64-бит. веществ.	64-бит. целый	32-бит. веществ.	32-бит. целый
x86_64, SSE2/3/4	2	2	4	4
x86_64, AVX	4	-	8	-
x86_64, AVX2	4	4	8	8
x86_64, AVX-512	8	-	16	-

- В современных графических процессорах (видеокартах) одновременно выполняется большое количество элементарных операций сразу над большими наборами пикселов
- В устройствах - ускорителях вычислений — над целыми и короткими вещественными
- В матричных регистрах емкости 1 килобайт процессоров Intel Xeon Scalable с системой команд AMX — для перемножения матриц из двухбайтных вещественных чисел

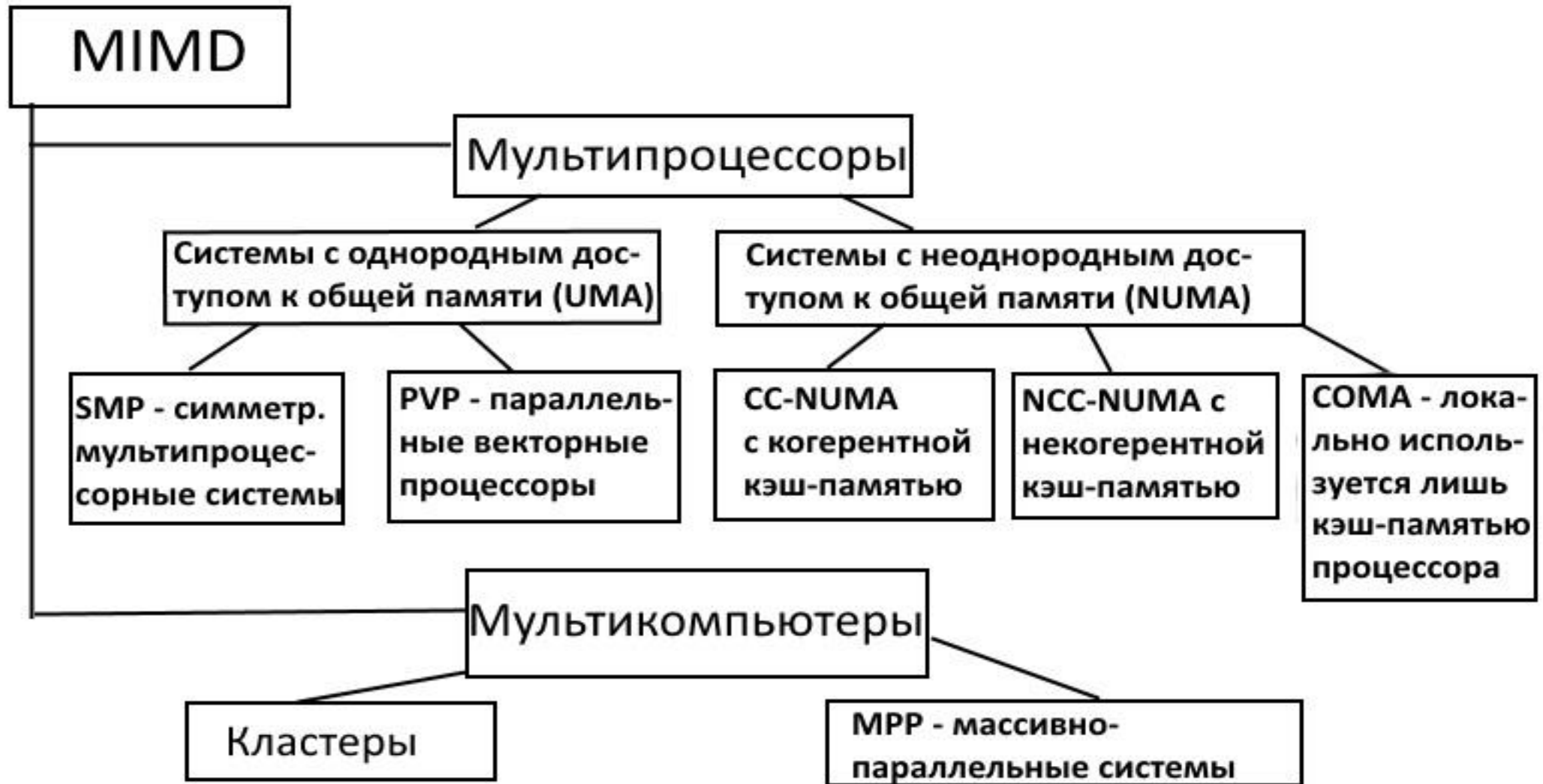
● **MISD** (Multiple Instruction Stream/Single Data Stream). Системы с многочисленными потоками команд и одиночными потоками данных.

- Системы с конвейерной обработкой данных. Например: обработка большого числа видеозаписей на последовательно соединенных вычислительных узлах

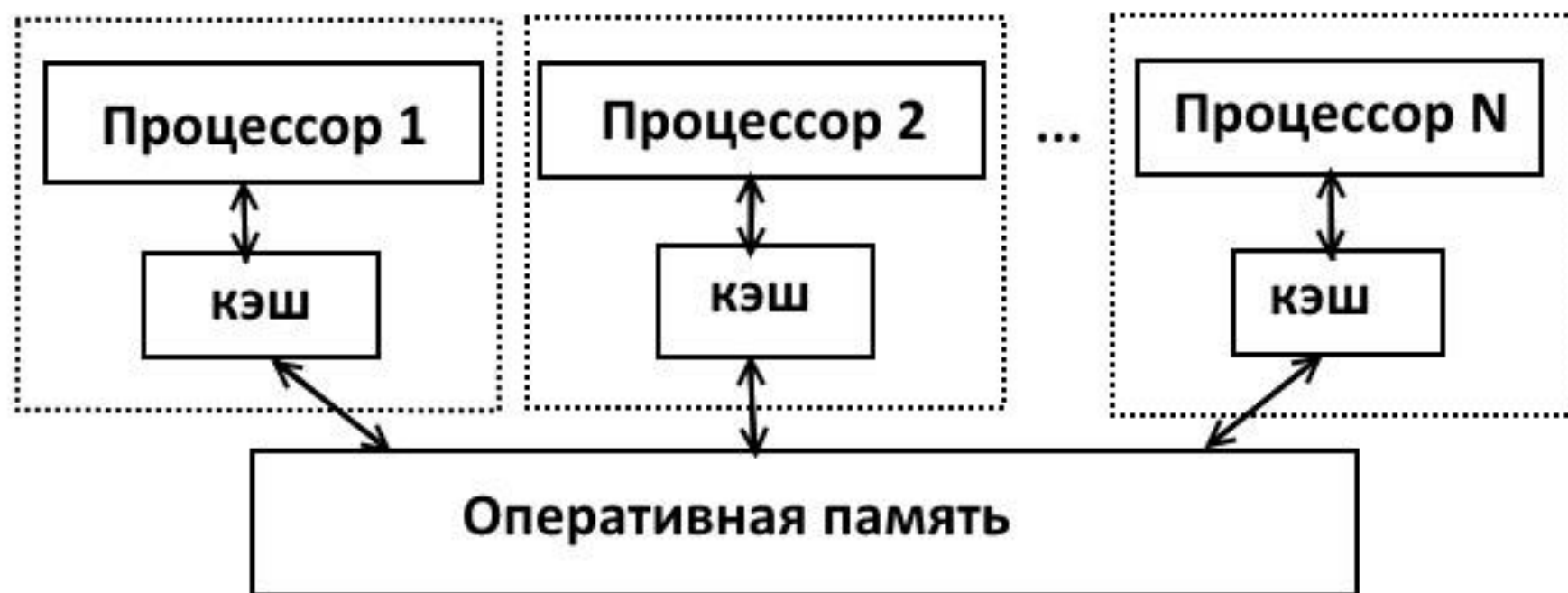


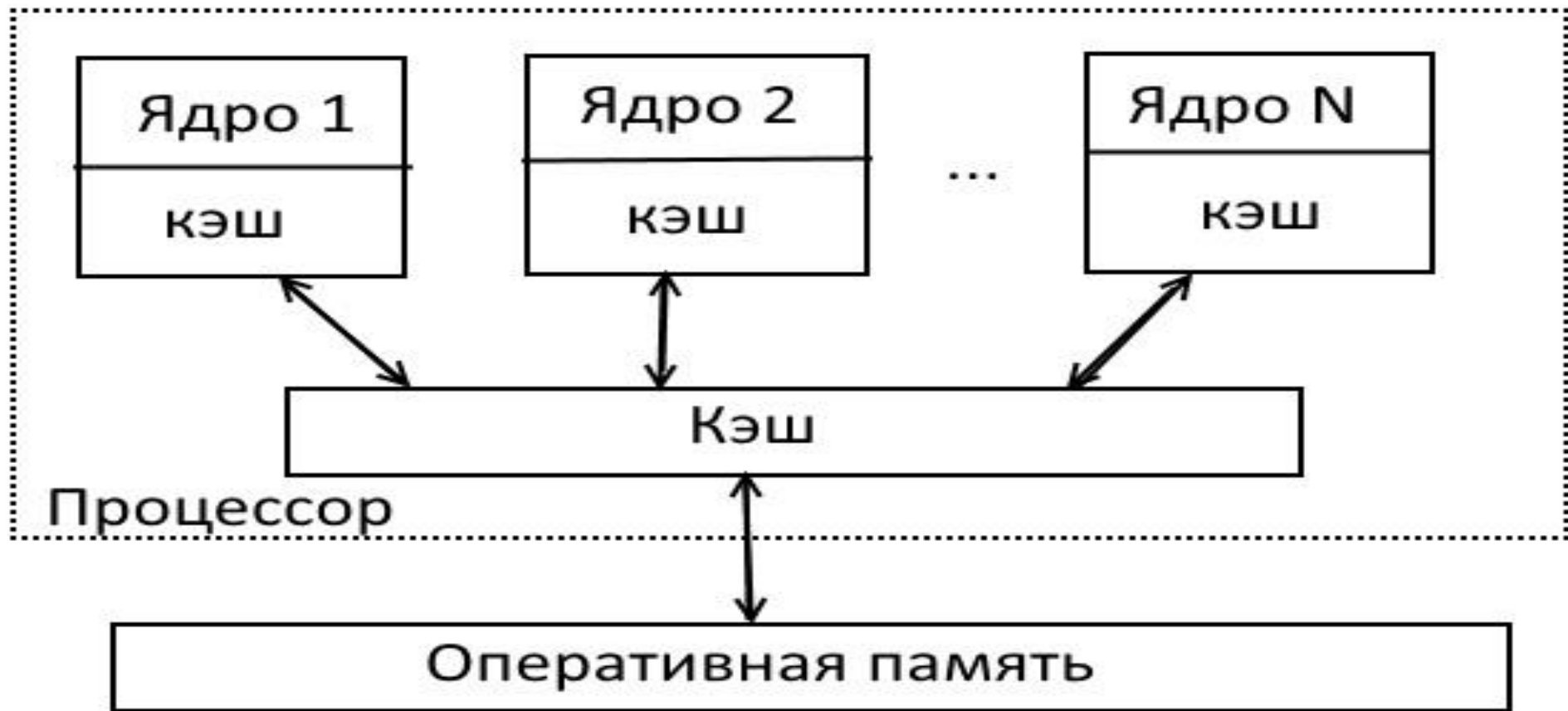
- Конвейерная модель вычислений современных процессоров: выборка, декодирование и непосредственное исполнение команд выполняются параллельно.
- Несколько пользователей в локальной компьютерной сети работают с одной и той же базой данных.

- **MIMD** (Multiple Instruction Stream/Multiple Data Stream). Системы с многочисленными потоками команд и многочисленными потоками данных.



- Различают два типа многопроцессорных систем — мультипроцессоры с общей разделяемой памятью, и мультимпьютеры, или системы с распределенной памятью.
- Реализуется асинхронная параллельная обработка данных
- Системы с однородным доступом к общей памяти (UMA — Uniform Memory Access).



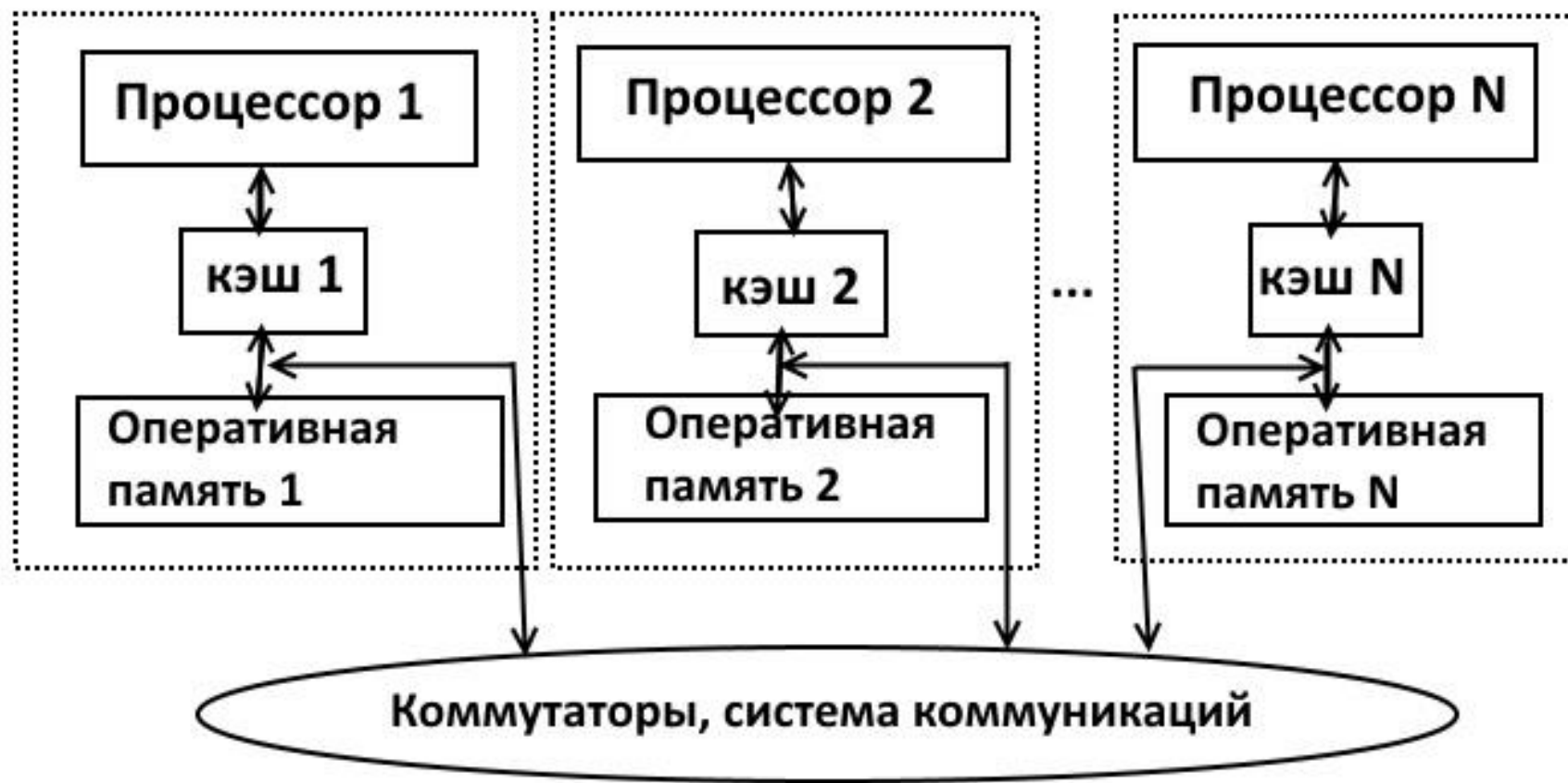


- Ранее применялись параллельные векторные процессоры — PVP, Parallel Vector Processors
- Сейчас: симметричные мультипроцессорные системы с общей памятью (SMP – Symmetric MultiProcessors).

- В симметричной мультипроцессорной системе с общей памятью (SMP) каждый процессор или отдельное ядро процессора характеризуется равноправным доступом ко всей оперативной памяти, и скорость доступа примерно одинакова для всех диапазонов адресов физической оперативной памяти
- Рабочие станции, серверы, настольные компьютеры, ноутбуки, сотовые телефоны
- Не критичен «переброс» потоков на другие процессоры (ядра) при их переключении операционной системой
- Технологии параллельного программирования: OpenMP, TPL .NET, потоки Java: основаны на многопоточности ОС.
- Проблема: доступ разных процессоров к общим данным и обеспечение в связи с этим однозначности (или когерентности) содержимого различных кэшей (cache coherence problem). Если все процессоры используют одну и ту же переменную, то ее копия окажется в кэш-памяти каждого из процессоров.

- Если какой-либо процессор обновит значение переменной, то потребуется ее обновить в кэш-памяти остальных процессоров.
- Это реализуется аппаратно: все копии переменной в кэш-памяти остальных процессоров отмечаются как недействительные, и дальнейшее обращение к переменной приведет к ее считыванию из оперативной памяти (или из кэш-памяти следующего уровня).
- Другая проблема: большая, но ограниченная пропускная способность тракта «процессор-оперативная память» затрудняет неограниченное повышение числа ядер в мультиядерных процессорах.

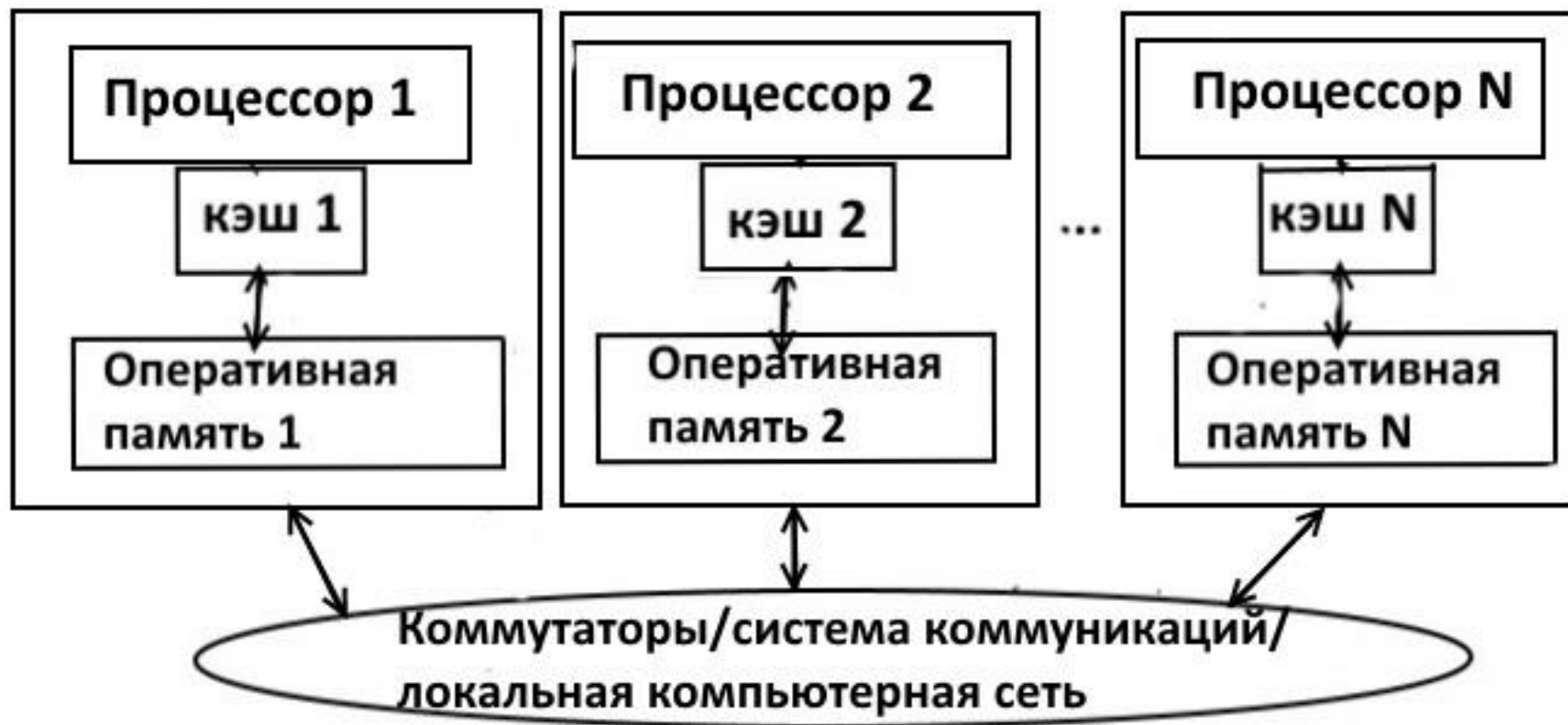
- Системы с неоднородным доступом к памяти (NUMA – Non-Uniform Memory Access) обеспечивают общий доступ к памяти в случае, когда оперативная память является физически распределенной.



- На аппаратном уровне с использованием коммутаторов и системы коммуникаций все блоки локальной для процессоров оперативной памяти помещаются в единое физическое адресное пространство.

- Системы, где аппаратно обеспечивается когерентность локальных кэш-памятей разных процессоров (Cache Coherent NUMA);
- Системы без аппаратной поддержки когерентности локальных кэш-памятей разных процессоров (Non Cache Coherent NUMA);
- Системы с представлением данных лишь локальной кэш-памятью процессоров (COMA - Cache-Only Memory Architecture)
- Проблема: доступ каждого процессора для диапазона физических адресов, соответствующих его локальной памяти, реализуется на порядок быстрее, чем для остальных процессоров.
- Для подобных систем желательно, чтобы каждый из процессоров обрабатывал данные в основном в своей локальной памяти
- Это приводит к необходимости закрепления потоков операционной системы за тем или иным процессором.
- Для всех систем с общей памятью: если требуется изменение общих для всех потоков данных, то необходимо обеспечить взаимное исключение (mutual exclusion), чтобы эти изменения в любой момент времени мог выполнять лишь один поток.

- Мультикомпьютеры отличаются от мультипроцессоров тем, что данная архитектура не обеспечивает общего доступа ко всей имеющейся в системе оперативной памяти.



- Процессор имеет доступ лишь к своей оперативной памяти, размещенной в текущем узле.

- Для доступа к данным на другом узле, либо для синхронизации процессов на другом узле мультимпьютера выполняются операции передачи сообщений (Message parsing operations).
- Компьютерные кластеры обычно формируются из узлов, представляющих собой отдельные компьютеры, либо конструируются из типовых компьютерных элементов. Для передачи сообщений между узлами кластера используется быстрая локальная компьютерная сеть (InfiniBand, MyriNet и т.д.)
- Число узлов в компьютерных кластерах может составлять несколько десятков тысяч. Все известные суперкомпьютеры представляют собой преимущественно кластерные системы.
- Технология параллельного программирования: MPI, основана на передаче и приеме сообщений.
- Отличие остальных массивно-параллельных вычислительных архитектур от компьютерных кластеров заключается в использовании специфического оборудования (коммутаторов) в узлах и при организации коммуникационной среды.

- Взаимодействие узлов кластера при помощи передачи сообщений по компьютерной сети обычно приводит к большим временным задержкам. Это накладывает дополнительные ограничения на тип разрабатываемых параллельных алгоритмов и программ.
- В современных кластерных системах узлы представляют собой симметричные мультипроцессорные системы с общей памятью. Используется «двухслойное» распараллеливание на MPI и OpenMP
- Узел современных кластерных систем может укомплектовываться несколькими устройствами-ускорителями вычислений (NVIDIA). Это на порядок повышает производительность узла в основном при операциях с двухбайтными вещественными (и его стоимость).
- Основная проблема: оптимизация обмена данными между основной памятью и памятью ускорителей
- Для преодоления этого: в узлах кластера могут использоваться мультитядерные процессоры Intel Xeon Scalable с системой команд AMX – для перемножения матриц из двухбайтных вещественных чисел в регистрах емкости 1 килобайт